

Jest

Wasaby предоставляет возможность покрывать код интерфейсных модулей тестами с помощью фреймворка [Jest](#).

Хранение тестов

1. Для хранения тестов нужно создать отдельный модуль (папка, содержащая одноимённый файл с расширением .s3mod). Имя модуля задается в виде <ИмяТестируемогоМодуля>Unit. Например, для модуля Controls модуль с тестами должен называться **ControlsUnit**.
2. Заполнить список его зависимостей, как минимум, там должен быть указан тестируемый модуль.
3. Добавить тег <test environment="NodeJS" />. Опция environment отвечает за окружение по умолчанию и поддерживает два значения:
 - NodeJS - тесты запустятся в окружении Node.js, браузерное API (window, document и т.п.) будет недоступно;
 - JSDOM - тесты запустятся в окружении [JSDOM](#), т.е. в эмуляции браузера. Большинство механизмов, доступных в настоящем браузере, будут доступны и здесь.
4. В теге <test> задайте параметр **framework**, в качестве значения укажите название фреймворка - **Jest**.

▼ Пример s3mod-файла для модуля с тестами

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <ui_module id="1234" name="<ИмяТестируемогоМодуля>Unit"
3     responsible="Иванов И.И." version="1.00">
4     <description/>
5     <depends>
6         <module id="0000" name="<ИмяТестируемогоМодуля>" />
7     </depends>
8     <load_after/>
9     <test environment="NodeJS" framework="Jest" />
10    <parameters/>
11    <resources/>
12    <preload/>
13 </ui_module>
```

5. Все тесты должны храниться в этом интерфейсном модуле в файлах с расширением .test.ts.

Пример простейшего теста

Допустим, у нас есть файл add.ts из интерфейсного модуля Example:

▼ Пример кода

```
1 export function add(a: number, b: number): number {
2     return a + b;
3 }
```

Чтобы его протестировать нужно в интерфейсном модуле ExampleUnit создать файл add.test.ts со следующим содержимым:

▼ Пример кода

```
1 import { add } from 'Example/add';
2
3 describe('add', () => {
4     it('adds 1 + 2 to equal 3', () => {
5         expect(add(1, 2)).toBe(3);
6     });
7 });
```

7 | });

Пример простейшего теста на React

▼ Пример кода

```
1  /**
2   * @jest-environment jsdom
3   */
4
5  import {render} from '@testing-library/react';
6  import { App } from 'JestTestingUtils/App';
7
8  it('test', () => {
9      render(<Component />, {
10         // Эта обёртка инициализирует системы Wasaby, например,
11         // систему событий и систему фокусов
12         wrapper: App
13     });
14 });
```

Полезные флаги при запуске тестов

1. [runInBand](#) - запускает все тесты в одном процессе для удобства отладки (по умолчанию каждый тестовый файл запускается в отдельном процессе).
2. [testPathPattern](#) - позволяет запускать тесты только из файлов, подходящих под регулярное выражение. Обратите внимание, что путь должен указываться до собранного js-файла.
3. [testNamePattern](#) - позволяет запускать только те тесты, полное название которых подходит под регулярное выражение. Полное название складывается из названия самого теста и всех блоков describe над ним.

Вспомогательные библиотеки

Для удобства написания тестов подключаются следующие библиотеки:

- [React Testing Library](#) помогает вам писать тесты, не зависящие от деталей реализации.
- [User event](#) помогает более удобно генерировать пользовательские события.

Основные принципы при написании тестов

Тесты пишутся ради того, чтобы они падали

Тесты, которые не падают при ошибках, бесполезны.

Если вы пишете тесты при исправлении ошибки, то соблюдать этот принцип очень просто - откатите фикс ошибки и проверьте падает ли тест. Если не падает - тест написан плохо.

Если вы пишете тесты при разработке нового функционала - либо пишете тесты до разработки функционала, либо пробуйте намеренно их ломать.

Один кейс - один тест

Если засовывать пачку кейсов в один it, то ошибку будет видно только для первого упавшего кейса. Т.е. для исправления всех ошибок придётся идти по алгоритму:

1. Отладил кейс.
2. Исправил ошибку.
3. Свалился следующий кейс.
4. Возвращаемся на шаг 1, пока остаются падающие кейсы.

В особо "хороших" случаях исправление позднего кейса может сломать один из предыдущих.

Даже подобные места лучше разбивать на отдельные тесты:

```
1 it('example', () => {
2   expect(toString(1)).toEqual('1');
3   expect(toString([1, 2])).toEqual('1,2');
4 });
```



Здесь не призывается писать одну проверку на тест. Если вам требуется проверить несколько вещей - делайте это, но они должны быть связаны.

По названию теста должно быть понятно что он проверяет

Многие названия тестов сводятся к следующему: название контрола.название метода.пара слов про входные данные, например:

```
1 describe('Controls/MyControl', () => {
2   describe('_onItemClick', () => {
3     it('myFlag: true', () => {
4       // ...сам тест
5       expect(notify).toHaveBeenCalledTimes(1);
6     });
7   });
8 });
```

На момент написания теста это кажется логично и понятно, но со временем меняется поведение и notify теперь вызывается два раза. Тест падает, и сразу возникает вопрос - а не ошибка ли это?

Если ошибка, то тест нужно править. Если не ошибка, то, скорее всего, вы добавили новый кейс, и нужно добавлять тесты. Если вы автор первоначального теста и помните свою логику - вам повезло. Если нет - придётся тратить время на разбор.

Варианты исправления разные. Если ожидаемое поведение можно коротко описать, то лучше вставить его в название. Если описание получается слишком длинным, можно хотя бы добавить комментарий рядом с тестом.

Тесты должны проверять публичное API

Если вы пишете тест на приватный метод, то вы встретитесь со следующими проблемами:

1. Вам все равно придётся писать тест на публичный метод. Да, в нём можно будет замочать приватный и проверять только аргументы вызова, но часто это лишняя возня.
2. При рефакторинге вам придётся идти и чинить тесты, даже если поведение вообще не изменилось.
3. Если ваш приватный метод доступен из тестов, то он доступен и из всего остального кода, а значит он не такой уж приватный.

То же самое касается приватных полей.



Понятно, что большинство существующих тестов завязаны на приватные поля/методы по необходимости, потому что нельзя было создавать реальные контролы. При переписывании на React такой подход постепенно должен уходить.

Тесты должны быть независимы от других тестов

Если тест полагается на какую-то глобальную переменную или на то, что инстанс будет в определённом состоянии после предыдущего теста, этот тест плохой:

1. Падение одного теста может по цепочке уронить все тесты после него.
2. Нельзя запускать тесты по отдельности, а значит разбор упавших тестов становится сложнее.
3. Править тесты становится сложнее, т.к. любая правка может непредсказуемо сломать что-то в другом тесте.



Если вам нужно делать какие-то одинаковые действия перед тестами - вынесите их в блок beforeEach или

отдельную функцию.

Тесты должны быть независимы от других модулей

Юнит-тест должен тестировать работу вашего модуля, а не вашего модуля+всех его зависимостей, поэтому всё ненужное должно заменяться на заглушки через `jest.spyOn(object, 'method').mockImplementation()`. По необходимости, можно проверять, что сторонний метод был вызван определённое количество раз или с определёнными параметрами (`expect(spy).toHaveBeenCalledTimes(number)` и `expect(spy).toHaveBeenCalledWith(arg[1], arg[2], ...)`).

Это позволит убрать чужие побочные эффекты типа вызовов `console.log`, а также ускорит прохождение тестов.

Тесты должны чистить за собой даже при падении

Довольно часто встречается подобный код:

```
1 it('example', () => {
2     // сам тест, его проверки и т.д.
3
4     sandbox.restore();
5 });
```

Если тест упадёт - `sandbox.restore()` никогда не будет вызван, что может сломать остальные тесты. Очистка должна быть в блоке `afterEach()`, а в идеале вообще не должна требоваться.



Все моки, сделанные через `jest.spyOn()` автоматически возвращаются к исходным значениям после каждого теста.

Асинхронные тесты не должны зависать при ошибках

Некоторые асинхронные тесты пишутся таким образом:

```
1 it('example', (done) => {
2     asyncMethod().then((result) => {
3         // проверка result
4         done();
5     });
6 });
```

Если `asyncMethod()` упадёт или `result` окажется не тем, чем должен, то `done` никогда не будет вызван, соответственно тест упадёт по таймауту. Обычно в логах при этом ничего не будет, кроме сообщения о том, что тест упал по таймауту.

Исправить ситуацию довольно легко - из теста можно вернуть Promise, и тогда Jest сам навесит на него обработчик ошибок. Ещё лучше переписать тест на `async/await`, тогда даже возвращать ничего не придётся.

Тесты не должны зависеть от реального времени

Иногда требуется работать с датами, или подождать пока вызовется коллбек `setTimeout/setInterval`. Не нужно добавлять ожидание в тест, можно просто подменить таймеры через [jest.useFakeTimers\(\)](#). После этого вы сможете "управлять временем" через [jest.advanceTimersByTime\(msToRun\)](#) и подобные методы.

Используйте встроенные функции для создания моков, вместо написания своего кода

Встречается такой код:

```
1 it('example', () => {
2     let isCalled = false;
3     instance.method = () => {
4         isCalled = true;
5     };
6 });
```

```
6 // ...тест
7 expect(isCalled).toBe(true);
8 });
```

При падении теста вы получите не очень понятное сообщение об ошибке в духе "ожидалось true, а получили false". Также, если вы захотите проверить что-то более сложное, например, аргументы при первом вызове, аргументы при втором вызове и т.д., то придётся городить ещё более объёмную пачку кода.

Можно переписать это так:

```
1 // mockImplementation добавлен для того, чтобы исходный метод не
  вызывался
2 jest.spyOn(instance, 'method').mockImplementation();
3 expect(instance.method).toHaveBeenCalled();
```

Тесты не могут проверить переводы

Функция локализации `rk()` работает через плагин `require`, в котором грузятся словари. На Jest плагины отключены, поэтому невозможно произвести загрузку словарей для тестирования.

Делайте тесты максимально реалистичными

1. Мокайте только то, что вам действительно не нужно;
2. Не лазайте в детали реализации контрола;
3. В идеале тестировать только то, что экспортируется из библиотек;
4. Проверяйте то, что видит пользователь, т.е. DOM, а не состояние контрола.

Оборачивайте моки контролов в `jest.fn()`

Пример:

```
1 jest.mock('libName', () => {
2   controlName: jest.fn((props) => {
3     return <div>{props.children}</div>
4   });
5 })
```

Такие моки позволяют вам и делать снапшот того, что вас реально интересует, и просто проверять что компонент был вызван с определёнными опциями.

Чем меньше снапшот, тем лучше

Не надо делать большие снапшоты по 100+ строк: их сложно проверять на правильность, они "хрупкие", т.к. обычно зависят от других компонентов, они плохо отражают, что проверяется.

Уменьшить размер снапшота можно следующими способами:

1. Подмена сторонних модулей через `jest.mock()`;
2. Снятие снапшота не всей страницы, а только интересующего элемента.

Давайте названия снапшотам

Если в тесте несколько снапшотов – давайте им названия, чтобы при падении тестов было проще их различать. Также название помогает описать то, что вы хотите проверить, даже если снапшот всего один.

Не используйте динамические элементы в снапшотах

Если внутри вашего снапшота есть динамические элементы (например, даты или случайно генерируемые `id`), то либо делайте их стабильными, либо удаляйте перед сохранением снапшота.

Не оборачивайте всё в `act()`

При использовании методов из **react-testing-library** и **user-event** оборачивание в **act()** происходит автоматически. Если у вас падают предупреждения, что что-то не обернуто в **act()** – скорее всего, в контроле что-то происходит асинхронно. В таких случаях можно воспользоваться **waitFor** или **waitForElementToBeRemoved**, в зависимости от того, что происходит асинхронно.

Избегайте поиска элементов напрямую через DOM

Вместо **querySelector**, **getElementsBy*** и т.п. лучше используйте методы из **react-testing-library**.

Лучше **getByText** или **getByRole**, в крайнем случае **getByTestId**. Эти методы выдают хорошие сообщения об ошибках и лучше передают намерения. Также поиск по тексту/роли/специальному атрибуту делает тесты более устойчивыми к изменениям.

Генерируйте события через user-event

Возьмём простой вариант – клик по элементу. Возникает желание просто позвать **element.click()**, но ведь реальные пользователи так не делают. Они сначала наводят мышку на элемент, потом нажимают кнопку, а потом отпускают. Это минимум три события, которые этот метод не сгенерирует.

Если взять вариант сложнее – ввод с клавиатуры, то там всё ещё хуже – на каждую кнопку пачка событий, между которыми ещё и задержка должна быть.

Не используйте таймауты для ожидания асинхронных действий

Таймауты использовать не рекомендуется, т.к. большинство людей забывают обрабатывать ошибки, и тест падает некорректно.

Вариантов обхода проблемы много:

1. **findBy*** из **react-testing-library**;
2. **waitFor**;
3. **waitForElementToBeRemoved**;
4. Ожидание **Promise**;
5. Фейковые таймеры (**jest.useFakeTimers()**).

Как подключить тесты к merge request?

1. Заливаем тесты в наш репозиторий.
 2. Пишем поручение [Сергею Сахаревичу](#) (для платформенных репозиториях - [Юрию Князеву](#)) с формулировкой:
 - *Прошу собрать в нашем репозитории тесты для интерфейса, запускаемые при пуше:*
 - Адрес проверяемого репозитория: ...
 - Путь до директории с тестами: ...
 - Путь до **CMakeLists**: ...
 3. Пишем поручение ответственному за сборку вашего приложения с формулировкой:
Прошу включать автозаворот добросок при файле тестов по веткам
- Подробнее о запуске тестов описано [здесь](#).